Real-Time Parallel Log / Event Triage Engine

A Regex-to-DFA Compiler and Lock-Free Multi-Core Pipeline for Streaming Log Classification

Adithya Santhosh P
Reg. No. 24BCI0099

Anvesha Tripathi
Reg. No. 24BCT0055

Team 1, Compiler Design Course Project, VIT · Progress report, October 2026

***Abstract*—Servers emit thousands of log lines per second, and the few that matter—repeated failed logins, injection attempts, crashes, leaked card or national-ID numbers—must be found as the lines arrive. This report presents the design and the current implementation of a real-time triage engine that classifies each log line into five categories (authentication failure, security threat, error, performance warning, personal data) with a severity and a confidence score. The core is a purpose-built regular-expression compiler: 26 detection rules are parsed by a recursive-descent parser, converted to a Thompson NFA and compiled by subset construction into one combined DFA, so every line is matched against all rules in a single linear pass. Checksum validators, a noisy-OR confidence model, PII masking and a sliding-window correlator are layered on the matcher, and a pipeline of threads connected by lock-free rings scales the engine across cores. The compiled DFA agrees with the C library's regex engine on all test lines and 100,000 generated strings, and parallel output is identical to sequential output. A faithful reproduction of the base paper's process-per-pattern design is also reported. At this milestone the compiler, the classification logic, the correlator and the parallel runtime are complete (35 of 45 planned tasks); benchmarking and accuracy evaluation remain.**

***Index Terms*—**regular expressions, finite automata, subset construction, log classification, intrusion detection, lock-free queues, multi-core processing.

I. INTRODUCTION

Modern services produce logs faster than people can read them. Hidden in the stream are lines that need action: repeated authentication failures, SQL-injection or path-traversal requests, application crashes, slow queries, and personal data such as card or Aadhaar numbers that must not be stored in clear. Triage—assigning each line a category, a severity and a confidence, and routing urgent lines to an alert stream—has to keep up with the input rate.

The base paper for this course project [5] classifies short text inputs by running one regular expression per process on separate cores. Our measurements (Section VI) show that this design is dominated by process-creation cost. This work keeps the goal—parallel regex classification—but changes the method: all rules are compiled into one deterministic automaton, and parallelism is applied across input lines rather than across patterns. The contributions are:

1) a regex compiler (lexer, recursive-descent parser, Thompson NFA, subset-construction DFA) restricted to the regular POSIX-ERE subset, checked against the C library's engine;
2) a classification layer with checksum validators, noisy-OR confidence, multi-label output and masking of personal data;
3) a correlator that detects multi-line attacks (brute force, password spraying) in sliding time windows;
4) a multi-threaded pipeline with lock-free single-producer/single-consumer (SPSC) rings; and
5) a faithful reproduction and measurement of the base paper's design.

II. BACKGROUND AND RELATED WORK

A. From Regular Expressions to Automata

Thompson's construction [1] converts a regular expression to an NFA with ε-transitions; subset construction [2] converts the NFA to a DFA whose states are sets of NFA states. A DFA reads one input symbol per table lookup, so matching takes time linear in the input with no backtracking, at the price of a possibly exponential number of states. Combining many rules into one automaton makes the per-line cost independent of the number of rules.

B. The Base Paper

The base paper [5] classifies short strings as e-mail, phone number, name, question or greeting using five regular expressions, one per forked process, and reports a 2.4× speed-up over a sequential implementation and near-linear scaling to six cores. Because the paper is an unverified student work, we treat its claims as hypotheses and test them (Section VI-C).

C. Log Data and Detection Practice

Loghub [3] provides real system logs; we use its OpenSSH, Apache, Linux, BGL and HDFS samples. Tools such as fail2ban [6] flag an address after a number of failures within a time window; our correlator uses the same parameter names (*maxretry*, *findtime*).

III. SYSTEM ARCHITECTURE

The system has a start-up phase and a run-time phase. At start-up the rule file is compiled into read-only DFA tables (Fig. 1). At run time a pipeline of threads streams lines through matching, classification, correlation and output (Fig. 2).

The reader splits the input into lines, decodes URL escapes for matching, parses timestamps and fills batches (256 lines by default). The dispatcher hands each batch to a worker, round-robin or to the least-loaded one. Workers share the DFA tables without locks; each runs the matcher and, for matched lines only, the validators, entity extraction and PII masking. The aggregator computes labels and confidence; the correlator

restores input order and applies the time windows; the router writes three JSON-lines sinks and prints a status line every second. Stages are connected by SPSC rings, and finished batches return to the reader through a free ring so that the steady state allocates no memory.

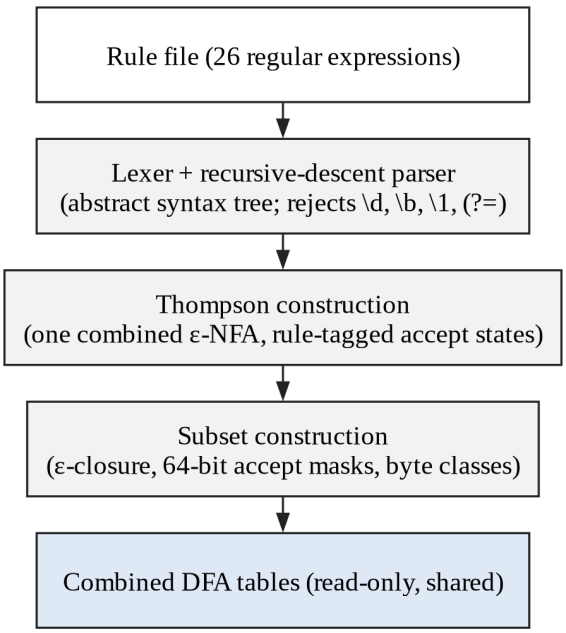


Fig. 1. Rule compilation, performed once at start-up.

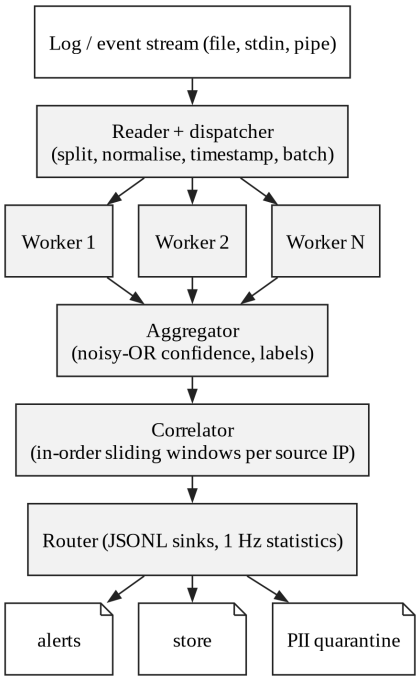


Fig. 2. Run-time pipeline. Arrows between threads are lock-free SPSC rings; the DFA tables are shared read-only by all workers.

IV. IMPLEMENTATION

A. Regex Front End

The grammar is the POSIX-ERE subset: literals, `.`, bracket expressions, `* + ? {n,m}`, alternation, grouping and the line anchors `^ $`. A lexer feeds a recursive-descent parser with one function per non-terminal. Character classes are 256-bit sets, so case-insensitive rules fold the set. Constructs outside the regular subset (shorthand classes, word boundaries, backreferences, lookaround, lazy quantifiers) are rejected with their position instead of being approximated; this guarantees linear-time matching and identical semantics to the C library.

B. NFA and DFA Construction

Each AST node maps to a Thompson fragment; the 26 rules are joined under one start state and each accept state carries its rule number (1,762 NFA states). Subset construction uses an explicit-stack ε-closure and a hash table of state sets. Every DFA state stores a 64-bit accept mask whose bit r is set when rule r has matched. To find matches anywhere in a line, the start set is added at every step; anchored rules are handled with assertion edges. Transitions are indexed by byte class. When the state count would exceed a cap, the rules are split into groups and one DFA is built per group; the default rules give 2 groups and 7,593 states in total.

C. Validation and Classification

A regular expression cannot tell a card number from any other 16-digit string, so matched spans are validated: the Luhn checksum [7] for cards, the Verhoeff checksum [8] for Aadhaar numbers, and a format check for PAN. A rule whose validator fails on every candidate span is discarded. Each surviving rule r receives a score

$$s_r = min(1,\ b_r + 0.25 \cdot v_r + 0.15 \cdot k_r) \quad (1)$$

where b_r is the rule's base precision, $v_r = 1$ if it has a validator that passed and $k_r = 1$ if a context keyword is present. The confidence of a category with surviving scores $s_1 \ldots s_m$ combines them by noisy-OR:

$$c = 1 - (1 - s_1)(1 - s_2)\ldots(1 - s_m) \quad (2)$$

A category becomes a label when $c \geq \tau = 0.5$, so a line can carry several labels. The primary label is the most severe one, ties being broken by confidence and then by a fixed order. Rules with a weak base (for example a bare HTTP 401) need a second signal to reach τ.

D. PII Masking

Before any output is written, validator-passed values are masked in the line: cards as `XXXX…1111`, Aadhaar as `XXXX XXXX 1234`, PAN as `XXXXX1234X`, secrets as `key=***`, plus e-mail addresses and phone numbers. Masked text goes to every sink, including the alert sink.

E. Correlator

Some attacks are invisible in one line. The correlator keeps, per source IPv4 address, a ring of the last 64 failures (time and user) in an open-addressing hash map, and works on log timestamps. Table I lists the patterns. An escalation raises the category confidence to 0.95, sets a flag, and is emitted once per window.

TABLE I
CORRELATION PATTERNS

Pattern	Trigger	Result
Brute force	≥ 5 failures, one IP, 60 s	SECURITY, HIGH
Password spray	≥ 5 distinct users, one IP, 300 s	SECURITY, HIGH
Fail then success	login from an IP currently in brute force	SECURITY, CRITICAL
Error storm	≥ 100 ERROR lines, 60 s	ERROR raised to HIGH

F. Parallel Runtime

An SPSC ring is a power-of-two array with a head index written only by the consumer and a tail index written only by the producer, each on its own 64-byte cache line. The producer

publishes a slot with a release store of the tail; the consumer observes it with an acquire load, so no locks or read-modify-write operations are needed. Batches are preallocated in a pool and recycled through a free ring. A full ring blocks the stage before it (back-pressure) or, with `--drop`, drops and counts batches. Workers can be pinned to separate physical cores. Since workers finish batches out of order, the correlator releases batches in input order, which makes the output independent of the worker count. End of input travels as a sentinel through every stage, and a shared stop flag unblocks all threads when a stage fails.

V. PROJECT STATUS

Work follows an ordered plan in which every task has a testable completion criterion; a correct single-threaded baseline was built first, and each later stage was compared with the output of the simpler one before it. Table II gives the status; Table III the module-wise split.

TABLE II
STATUS BY PHASE

Phase	Scope	Status
0	Build system, tests, sample logs	Done
1	Single-thread baseline (C regex library)	Done
2	Regex compiler and DFA engine	Done
R	Base-paper reproduction (6 claims)	Done*
3	Validators, confidence, PII masking	Done
4	Parallel runtime	Done
5	Correlator	Done
6	Benchmark tooling	Pending
7	Evaluation on full datasets	Pending

*One optional task (conflict-resolution scoring) is not done. In total 35 of the 45 planned tasks are complete.

TABLE III
MODULES AND SOURCE SIZE

Module	Source files	Lines
CLI, driver	main, cli	321
Rule loader	rules	417
Input	reader, normalize, timestamp, batch	683
Regex compiler	regex_lex, regex_parse, ast, nfa, dfa, dot	2,305
Matching engines	engine, engine_dfa, engine_posix	312
Validation, extraction	validators, extract	500
Classification	aggregate	213
PII masking	mask	160
Correlator	correlate	327
Output	route, json	501
Parallel runtime	spsc, pool, worker, dispatch, pipeline, stats	1,354
Shared types	types	80
Engine total	29 source files (C11)	**7,173**

VI. VERIFICATION AND RESULTS

A. Correctness

Table IV summarises the checks. The DFA is compared with the C library's *regexec* per rule; classification and correlation are compared with an independent Python implementation of the same specification; parallel runs are compared with the single-worker run record by record.

TABLE IV
VERIFICATION

Check	Result
19 unit-test programs (incl. 10M-item ring stress test)	pass
DFA vs. regexec: corpus + 100,000 random strings	0 mismatches
C engine vs. Python reference, 10,000+ lines, 6 logs	all fields equal
1, 2, 4, 8 workers vs. 1 worker, 2 engines, 6 logs	72 runs equal
ThreadSanitizer, AddressSanitizer on tests and runs	no reports
Allocations at 100k vs. 500k input lines	constant

B. Attack Detection on a Real Log

On the Loghub OpenSSH sample (2,000 lines) the correlator raises 34 brute-force and 5 password-spray escalations; every escalated record appears in the alert sink and agrees with the Python reference.

C. Reproduction of the Base Paper

The paper's program was rebuilt as described (one *fork* per pattern per input, with *regcomp* and *regfree* in every child) and compared with sequential, thread-based and DFA variants on the same five patterns (5 runs per point, 14-core Intel i7-13650HX). Table V gives selected results; no value was tuned to match the paper.

TABLE V
BASE-PAPER DESIGN VERSUS ALTERNATIVES

Measurement	Result
fork design, time per input	0.19–0.21 ms (paper: 3.7 ms)
fork vs. sequential, recompiling	12–13× slower
fork vs. sequential, precompiled	about 500× slower
Peak speed-up, fork, 8 cores	1.72× (paper: 5.62×)
Peak speed-up, split by input, 6 cores	4.83×
Combined DFA, 6 cores, vs. sequential	5.29× (own scaling)

The per-pattern fork design is dominated by process creation (about 200 µs per input against well under 1 µs of matching) and does not reproduce the paper's claims; splitting the *inputs* across cores scales, which motivates the present design.

VII. REMAINING WORK AND CONCLUSION

Phase 6 adds the fork engine inside the triage binary, a benchmark mode with latency percentiles, a synthetic log generator with exact ground truth, rule sets of 5–64 rules and an experiment runner. Phase 7 evaluates throughput and scaling of the DFA, POSIX and fork engines, and measures accuracy on full datasets with a stratified sample labelled independently by both authors (precision, recall, F1, Cohen's κ, confidence calibration). Optional extensions are DFA minimisation and ordered output. No throughput or accuracy figures for the engine are reported here, because they will be taken only from saved benchmark runs.

The regex compiler, the classification logic, the correlator and the parallel runtime are implemented and verified, giving a working system that classifies, masks and correlates real log data on multiple cores with output identical to the sequential run.

REFERENCES

[1] K. Thompson, “Programming techniques: Regular expression search algorithm,” *Commun. ACM*, vol. 11, no. 6, pp. 419–422, 1968.

[2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[3] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, “Loghub: A large collection of system log datasets for AI-driven log analytics,” in *Proc. IEEE Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2023.

[4] *IEEE Standard for Information Technology—Portable Operating System Interface (POSIX)*, IEEE Std 1003.1-2017, Regular Expressions.

[5] “Parallel text classification system using regex pattern matching on multi-core architectures,” base paper provided by the course faculty, unpublished.

[6] fail2ban: intrusion prevention software. [Online]. Available: https://www.fail2ban.org

[7] H. P. Luhn, “Computer for verifying numbers,” U.S. Patent 2 950 048, Aug. 23, 1960.

[8] J. Verhoeff, *Error Detecting Decimal Codes*, Mathematical Centre Tracts 29. Amsterdam, The Netherlands: Mathematisch Centrum, 1969.